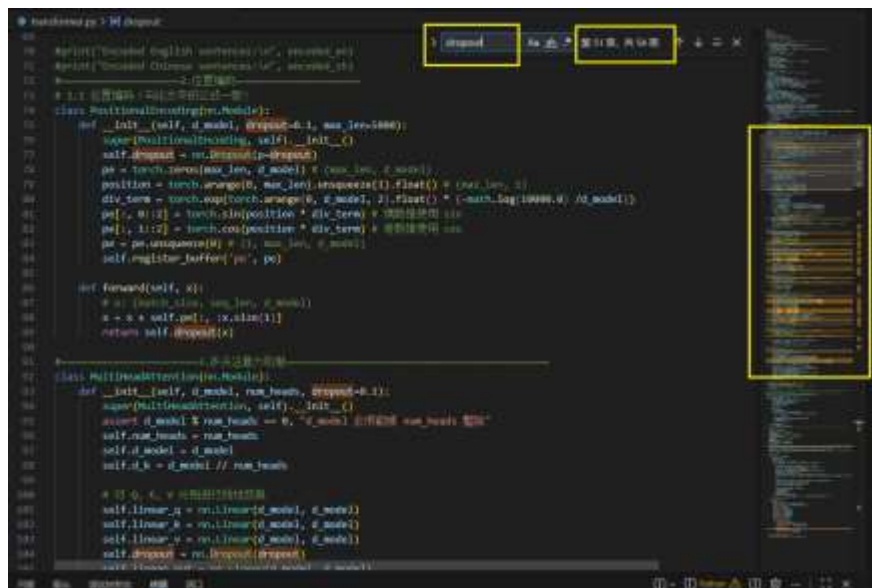


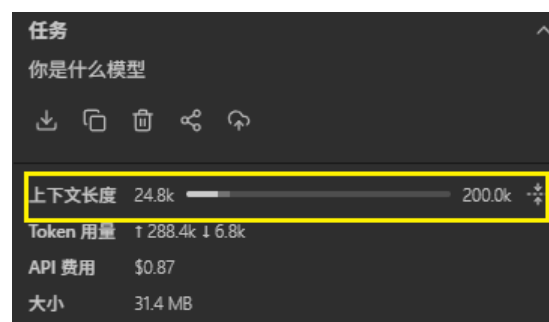
01. Transformer 中的哪些设计是用来规避模型过拟合问题？

大量使用的 Dropout。



02. Transformer 中对输入序列的长度有什么限制？

- 计算与序列 L 的二次方成正比。序列越长，成本爆炸。多头注意力（MHA）：需要计算「每个 token 与其他所有 token 的关联」——比如序列长度为 L（假设就是有 L 个 token），每个 token 要和 L 个 token 比对应关系，这就产生了 $L \times L$ 的“平方级”计算
- 工程上模型常有固定 context window（如 claude sonnet4.5 上下文窗口 200k）



- 如果用 Learned Absolute Position Embedding，通常有一个绝对长度上限 `max_position_embeddings`

03. 为什么说 Transformer 中的计算是并行化的？

- 自注意力机制的并行性：注意力机制通过矩阵运算一次性处理整个输入序列，而无需像 RNN 那样逐步计算（依赖于上一个时间步的 output 来作为下一个时间步的 input）。Transformer 输入序列通过线性变换并行生成 Query、Key 和 Value 矩阵，随后通过矩阵乘法并行计算注意力得分，并对 Value 矩阵进行加权求和。

- 多头注意力的并行性：多头注意力机制将自注意力拆分为多个独立的子空间，每个头独立计算 Q、K、V 矩阵，并在不同计算单元上并行执行。头与头之间无依赖关系，最终将结果拼接。这种设计不仅增强了模型的表达能力，还通过并行计算显著降低时间复杂度。

04. 没有一个模型是适用于所有问题的，在什么类型的任务或场景中可能会偏好其他模型而不是 Transformer？

答：

1) 超长序列、需要极低复杂度。用 transformer 算不划算或者难训练的时候。如处理高频时间序列、日志流、超长文档在线处理

1. 总复杂度 (合并MHA+FFN)

$$\text{总FLOPs} = (4Ld^2 + 2L^2d) + 8Ld^2 = 12Ld^2 + 2L^2d$$

2. 随 `seq_len(L)` 的增长趋势

- 模型维度 `d` 是固定值 (如768)，因此：
 - 次要项: $12Ld^2 \rightarrow$ 随 `L` 线性增长 ($O(L)$)；
 - 主导项: $2L^2d \rightarrow$ 随 `L` 平方增长 ($O(L^2)$)。

3. 结论

- Transformer Layer的计算复杂度为 $O(L^2d + Ld^2)$ ；
- 当 `seq_len(L)` 增加时， $O(L^2)$ 项主导复杂度增长，这是Transformer处理长序列 (如 `L>2048`) 时效率低下的根本原因。

2) 数据很少且结构强、先验很明确。如有具体的公式指导的工艺规律、物理建模仿真、工业控制等。

05.从可解释性的角度来分析 Transformer 在预测过程中每个模块的功能。

- Token 词向量化
- 位置编码：
- 多头 Self-Attention：每个位置根据当前表示 (Q) 去“检索”序列里其他位置的信息 (K,V)，形成上下文融合。多个头就是并行的多套“检索规则”
- 掩码多头自注意力：保证预测第 `t` 个 token 时只用到 $\leq t$ 的信息，符合生成逻辑。(不会算命式的知道用户还没说完的话)
- 交叉自注意力：Decoder 在生成时用自己的状态做 Query，去 Encoder 表示里找最相关的信息 (K,V)
- FFN：提升表达能力
- Residual + LayerNorm：稳定训练作用
- Linear + Softmax：输出概率，在词表上对每个词进行概率打分。

05. 尝试调整参数或扩充训练集来优化视频中模型训练效果。

单独摘出来了训练用 zh-en 句子对儿并进行了训练数量扩充，五组太少了，存到了 json 里面。

```

135
136
137     "zh": "那 不 错 。 "
138 },
139 {
140     "en": "I want to learn.",
141     "zh": "我 想 学 习 。 "
142 },
143 {
144     "en": "Can you help me?",
145     "zh": "你 能 帮 我 吗 ? "
146 },
147 {
148     "en": "Of course I can.",
149     "zh": "当 然 可 以 。 "
150 },
151 {
152     "en": "See you tomorrow.",
153     "zh": "明 天 见 。 "
154 },
155 {
156     "en": "Have a nice day.",
157     "zh": "祝 你 愉 快 。 "
158 },
159 {
160     "en": "I love you.",
161     "zh": "我 爱 你 。 "
162 },
163 ]
164 )
165

```

成功翻译一句话。

```

11  def translate(model, src_sentence, max_len=20):
12      for i in range(max_len):
13          tgt_tensor = torch.tensor(tgt_ids, dtype=torch.long).unsqueeze(0)
14          tgt_mask = generate_square_subsequent_mask(tgt_tensor.size(1)).to(tgt_tensor.device)
15          with torch.no_grad():
16              output = model(src_tensor, tgt_tensor, tgt_mask=tgt_mask)
17              next_token = output[0, -1, :].argmax().item()
18              tgt_ids.append(next_token)
19              if next_token == word2idx["<eos>"]:
20                  break
21      # 过滤掉特殊标记 <eos> 和 <unk>
22      translated_tokens = [idx2word_zh[idx] for idx in tgt_ids if idx > word2idx["<eos>"] and idx > word2idx["<unk>"]]
23      return " ".join(translated_tokens)
24
25  # 测试数据
26  test_sentences = ["I am a student."]
27
28  print("\n\n翻译测试")
29  print("-" * 60)
30  for example_en in test_sentences:
31      translated_result = translate(model, example_en)
32      print("待翻译句子: {example_en}")
33      print("翻译结果: {translated_result}")
34      print("-" * 60)
35

```

PS C:\XT\Code\week9 & "C:\Program Files\Python311\python.exe" c:\XT\Code\week9\translate.py
 模型加载成功!

翻译测试:

```

=====
待翻译句子: I am a student.
翻译结果: 我 是 一 名 学 生 。
=====

```

PS C:\XT\Code\week9>

尝试更复杂的，基本能翻译得差不多，对于部分没见过的词会乱说。

```

transformer.py transformer_torch.py transformer_data_loader.py transformer_model.py translator.py X
transformer.py > -
def translate(model, src_sentence, max_len=20):
    translated_tokens = [ids2word_th[100] for idx in tgt_ids if ids2word_th[100] not in ["<EOS>", "<EOS>"]]
    return " ".join(translated_tokens)

# 测试翻译
test_sentences = ["I am a student.",
                  "He loves programming.",
                  "The weather is nice today.",
                  "Can you help me?",
                  "This is a test sentence."]

print("\n测试翻译:")
print("-" * 60)
for example_en in test_sentences:
    translated_result = translate(model, example_en)
    print(f"待翻译句子: {example_en}")
    print(f"翻译结果: {translated_result}")
    print("-" * 60)

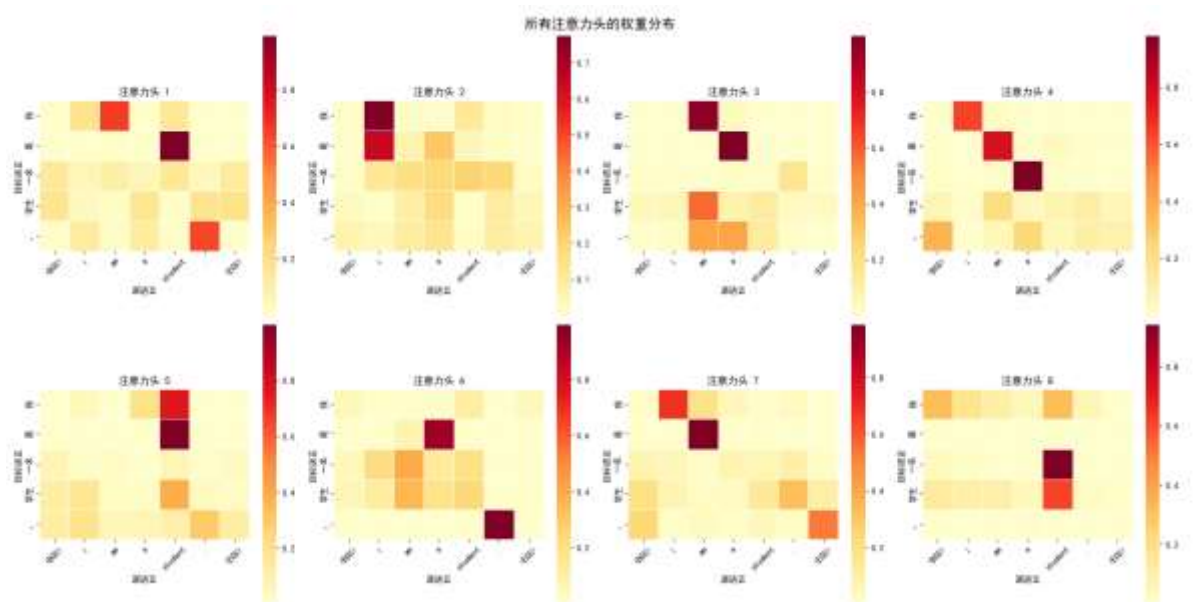
PS C:\TZ\Code\week8 & "C:\Program Files\Python11\python.exe" c:\TZ\Code\week8\translate.py
SystemError: invalid character ' ' (U+FFFC)
PS C:\TZ\Code\week8 & "C:\Program Files\Python11\python.exe" c:\TZ\Code\week8\translate.py
模型加载成功!

测试翻译:
待翻译句子: I am a student.
翻译结果: 我是一名学生。
待翻译句子: He loves programming.
翻译结果: 他是一名学生。
待翻译句子: The weather is nice today.
翻译结果: 天气很好。
待翻译句子: Can you help me?
翻译结果: 你能帮我吗?
待翻译句子: This is a test sentence.
翻译结果: 这是一本书。

PS C:\TZ\Code\week8

```

06. 如果上题得到解决，可尝试提取并可视化 Transformer 模型中多头自注意力的权重。



07. 尝试使用 PyTorch 中 nn 模块的注意力层来重构视频中的训练过程

答: 见源代码

09. 课程示例主要采用逐句处理的方式进行训练。现在请设计一个数据加载模块，基于 PyTorch DataLoader 完成批量数据的处理。

要求:

- 编写 Dataset 类, 继承 `torch.utils.data.Dataset`, 将平行数据 (编码后的英文和中文句子) 作为内部数据。
- Dataset 类应返回一个 dict 或 tuple, 包含源句子和目标句子的 tensor。
- 使用 DataLoader 封装该 Dataset, 使得每个 batch 中自动进行 padding 并生成 batch 数据传给模型

答: 见源代码